Binary Search

Ques.

162. Find Peak Element

Solved

Medium | Topics | Companies

A peak element is an element that is strictly greater than its neighbors.

Given a **0-indexed** integer array `nums`, find a peak element, and return its index. If the array contains multiple peaks, return the index to **any of the peaks**.

You may imagine that `nums[-1] = nums[n] = -∞`. In other words, an element is always considered to be strictly greater than a neighbor that is outside the array.

You must write an algorithm that runs in `O(log n)` time.

Example 1:

```
Input: nums = [1,2,3,1]
Output: 2
```

C++ | Auto

```cpp
class Solution {
public:
    int findPeakElement(vector<int>& nums) {
        int n = nums.size();
        int low = 0;
        int high = n - 1;
        if(n==1)return 0;

        while (low <= high) {
            int mid = low + (high - low) / 2;

            if ((mid == 0 && nums[mid] > nums[mid + 1]) ||
                (mid == n - 1 && nums[mid] > nums[mid - 1]) ||
                (nums[mid] > nums[mid + 1] && nums[mid] > nums[mid - 1])) {
                return mid;
            } else if (mid < n - 1 && nums[mid] < nums[mid + 1]) {
                low = mid + 1;
            } else {
                high = mid - 1;
            }
```

Saved

Ques.

350. Intersection of Two Arrays II

Solved

Topics | Companies

Given two integer arrays `nums1` and `nums2`, return *an array of their intersection*. Each element in the result must appear as many times as it shows in both arrays and you may return the result in **any order**.

Example 1:

```
Input: nums1 = [1,2,2,1], nums2 = [2,2]
Output: [2,2]
```

Example 2:

```
Input: nums1 = [4,9,5], nums2 = [9,4,9,8,4]
Output: [4,9]
Explanation: [9,4] is also accepted.
```

C++ | Auto

```cpp
class Solution {
public:
    vector<int> intersect(vector<int>& nums1, vector<int>& nums2) {
        sort(nums1.begin(), nums1.end());
        sort(nums2.begin(), nums2.end());
        vector<int>nums;
        int i = 0;
        int j = 0;
        int n1 = nums1.size();
        int n2 = nums2.size();
        while(i<n1 && j<n2){
            if(nums1[i]==nums2[j]){
                nums.push_back(nums1[i]);
                i++;
                j++;

            }
            else if(nums1[i]<nums2[j]){
                i++;
            }
```

Saved

gmf Ques.

33. Search in Rotated Sorted Array

Solved

Medium | Topics | Companies

There is an integer array `nums` sorted in ascending order (with **distinct** values).

Prior to being passed to your function, `nums` is **possibly rotated** at an unknown pivot index `k` (`1 <= k < nums.length`) such that the resulting array is `[nums[k], nums[k+1], ..., nums[n-1], nums[0], nums[1], ..., nums[k-1]]` (**0-indexed**). For example, `[0,1,2,4,5,6,7]` might be rotated at pivot index `3` and become `[4,5,6,7,0,1,2]`.

Given the array `nums` **after** the possible rotation and an integer `target`, return *the index of* `target` *if it is in* `nums`, *or* `-1` *if it is not in* `nums`.

You must write an algorithm with `O(log n)` runtime complexity.

Example 1:

```
Input: nums = [4,5,6,7,0,1,2], target = 0
Output: 4
```

```cpp
int search(vector<int>& nums, int target) {
    int n = nums.size();
    int low = 0;
    int high =  n-1;
    while(low<=high){
        int mid = low + (high-low)/2;
        if(nums[mid]==target)return mid;
        else if(nums[mid]<nums[high]){
            if(nums[mid]<target && nums[high]>=target){
                low = mid+1;
            }
            else {
                high = mid-1;
            }
        }
        else{
            if(nums[mid]>target && nums[low]<=target){
                high = mid-1;
            }
            else{
                low = mid+1;
            }
        }
    }
    return -1;
}
```

C. Hamburgers

time limit per test: 1 second

memory limit per test: 256 megabytes

Polycarpus loves hamburgers very much. He especially adores the hamburgers he makes with his own hands. Polycarpus thinks that there are only three decent ingredients to make hamburgers from: a bread, sausage and cheese. He writes down the recipe of his favorite "Le Hamburger de Polycarpus" as a string of letters 'B' (bread), 'S' (sausage) и 'C' (cheese). The ingredients in the recipe go from bottom to top, for example, recipe "BSCBS" represents the hamburger where the ingredients go from bottom to top as bread, sausage, cheese, bread and sausage again.

Polycarpus has n_b pieces of bread, n_s pieces of sausage and n_c pieces of cheese in the kitchen. Besides, the shop nearby has all three ingredients, the prices are p_b rubles for a piece of bread, p_s for a piece of sausage and p_c for a piece of cheese.

Polycarpus has r rubles and he is ready to shop on them. What maximum number of hamburgers can he cook? You can assume that Polycarpus cannot break or slice any of the pieces of bread, sausage or cheese. Besides, the shop has an unlimited number of pieces of each ingredient.

Input

The first line of the input contains a non-empty string that describes the recipe of "Le Hamburger de Polycarpus". The length of the string doesn't exceed 100, the string contains only letters 'B' (uppercase English B), 'S' (uppercase English S) and 'C' (uppercase English C).

The second line contains three integers n_b, n_s, n_c $(1 \le n_b, n_s, n_c \le 100)$ — the number of the pieces of bread, sausage and cheese on Polycarpus' kitchen. The third line contains three integers p_b, p_s, p_c $(1 \le p_b, p_s, p_c \le 100)$ — the price of one piece of bread, sausage and cheese in the shop. Finally, the fourth line contains integer r $(1 \le r \le 10^{12})$ — the number of rubles Polycarpus has.

Please, do not write the %lld specifier to read or write 64-bit integers in C++. It is preferred to use the cin, cout streams or the %I64d specifier.

Output

Print the maximum number of hamburgers Polycarpus can make. If he can't make any hamburger, print 0.

```cpp
#include <bits/stdc++.h>
using namespace std;\
int main() {
    string st;
    cin >> st;
    long long nb, ns, nc;
    cin >> nb >> ns >> nc;
    long long pb, ps, pc;
    cin >> pb >> ps >> pc;
    long long r;
    cin >> r;
    long long b = 0, s = 0, c = 0;
    for (char ch : st) {
        if (ch == 'B') b++;
        else if (ch == 'S') s++;
        else c++;
    }
    auto canMake = [&](long long x) {
        long long need_b = max(0LL, b * x - nb);
        long long need_s = max(0LL, s * x - ns);
        long long need_c = max(0LL, c * x - nc);
        long long cost = need_b * pb + need_s * ps + need_c * pc;
        return cost <= r;
    };
    long long low = 0, high = 1e13, ans = 0;
    while (low <= high) {
        long long mid = (low + high) / 2;
        if (canMake(mid)) {
            ans = mid;
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    cout << ans << endl;
    return 0;
}
```

D1. Magic Powder - 1

time limit per test: 1 second

memory limit per test: 256 megabytes

This problem is given in two versions that differ only by constraints. If you can solve this problem in large constraints, then you can just write a single solution to the both versions. If you find the problem too difficult in large constraints, you can write solution to the simplified version only.

Waking up in the morning, Apollinaria decided to bake cookies. To bake one cookie, she needs n ingredients, and for each ingredient she knows the value a_i — how many grams of this ingredient one needs to bake a cookie. To prepare one cookie Apollinaria needs to use all n ingredients.

Apollinaria has b_i gram of the i-th ingredient. Also she has k grams of a magic powder. Each gram of magic powder can be turned to exactly 1 gram of any of the n ingredients and can be used for baking cookies.

Your task is to determine the maximum number of cookies, which Apollinaria is able to bake using the ingredients that she has and the magic powder.

Input

The first line of the input contains two positive integers n and k ($1 \le n, k \le 1000$) — the number of ingredients and the number of grams of the magic powder.

The second line contains the sequence $a_1, a_2, ..., a_n$ ($1 \le a_i \le 1000$), where the i-th number is equal to the number of grams of the i-th ingredient, needed to bake one cookie.

The third line contains the sequence $b_1, b_2, ..., b_n$ ($1 \le b_i \le 1000$), where the i-th number is equal to the number of grams of the i-th ingredient, which Apollinaria has.

Output

Print the maximum number of cookies, which Apollinaria will be able to bake using the ingredients that she has and the magic powder.

```cpp
bool canmake(long long x,long long k,vector<int>&a,vector<int>&b){
    long long sum = 0;
    int n = a.size();
    for(int i = 0; i<n; i++){
        long long need = 1LL*x*a[i]-b[i];
        if(need>0){
            sum += need;
        }
        if(sum>k)return false;
    }
    return sum<=k;

}
int main() {
    int n;
    long long k;
    cin>>n>>k;
    vector<int>a(n);
    for(int i = 0; i<n; i++){
        cin>>a[i];
    }
    vector<int>b(n);
    for(int i = 0; i<n; i++){
        cin>>b[i];
    }
    long long low = 0;
    long long high = 2e9;
    long long ans = 0;
    while(low<=high){
        long long mid = low + (high-low)/2;
        if(canmake(mid,k,a,b)){
            ans = mid;
            low = mid+1;
        }
        else{
            high = mid-1;
        }
    }
```

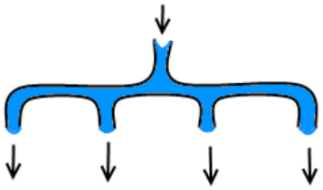
B. Pipeline

time limit per test: 0.4 seconds

memory limit per test: 256 megabytes

Vova, the Ultimate Thule new shaman, wants to build a pipeline. As there are exactly n houses in Ultimate Thule, Vova wants the city to have exactly n pipes, each such pipe should be connected to the water supply. A pipe can be connected to the water supply if there's water flowing out of it. Initially Vova has only one pipe with flowing water. Besides, Vova has several splitters.

A splitter is a construction that consists of one input (it can be connected to a water pipe) and x output pipes. When a splitter is connected to a water pipe, water flows from each output pipe. You can assume that the output pipes are ordinary pipes. For example, you can connect water supply to such pipe if there's water flowing out from it. At most one splitter can be connected to any water pipe.



The figure shows a 4-output splitter

Vova has one splitter of each kind: with $2, 3, 4, ..., k$ outputs. Help Vova use the minimum number of splitters to build the required pipeline or otherwise state that it's impossible.

Vova needs the pipeline to have exactly n pipes with flowing out water. Note that some of those pipes can be the output pipes of the splitters.

Input

The first line contains two space-separated integers n and k ($1 \le n \le 10^{18}$, $2 \le k \le 10^{9}$).

```
3   int main() {
4       long long n, k;
5       cin >> n >> k;
6       if (n == 1) {
7           cout << 0 << endl;
8           return 0;
9       }
10      // max tanks that can be filled
11      long long total = 1 + k*(k-1)/2; // (2-1)+(3-1)...
        +(k-2)+k, note that last one is k, as there will be
        not anyother splitter connected to the last one.
12      if (total < n) {  // even the best case can't fill
13          cout << -1 << endl;
14          return 0;
15      }
16      long long low = 0, high = k-1, ans = -1;  //(2-1),
        (3-1)....(k-1))
17      while (low <= high) {
18          long long mid = (low + high) / 2;
19          long long filled = total -((mid * (mid + 1) / 2));
20          if (filled >= n) {
21              ans = mid;
22              low = mid+1;
23          } else {
24              high = mid-1;
25          }
26      }
27      cout << (k - ans-1) << endl;
28      return 0;
```

C. Mike and Chocolate Thieves

time limit per test: 2 seconds
memory limit per test: 256 megabytes

Bad news came to Mike's village, some thieves stole a bunch of chocolates from the local factory! Horrible!

Aside from loving sweet things, thieves from this area are known to be very greedy. So after a thief takes his number of chocolates for himself, the next thief will take exactly k times more than the previous one. The value of k ($k > 1$) is a secret integer known only to them. It is also known that each thief's bag can carry at most n chocolates (if they intend to take more, the deal is cancelled) and that there were **exactly four** thieves involved.

Sadly, only the thieves know the value of n, but rumours say that the numbers of ways they could have taken the chocolates (for a fixed n, but not fixed k) is m. Two ways are considered different if one of the thieves (they should be numbered in the order they take chocolates) took different number of chocolates in them.

Mike want to track the thieves down, so he wants to know what their bags are and value of n will help him in that. Please find **the smallest possible** value of n or tell him that the rumors are false and there is no such n.

Input

The single line of input contains the integer m ($1 \le m \le 10^{15}$) — the number of ways the thieves might steal the chocolates, as rumours say.

Output

Print the only integer n — the maximum amount of chocolates that thieves' bags can carry. If there are more than one n satisfying the rumors, **print the smallest one**.

If there is no such n for a false-rumoured m, print -1.

```cpp
#include <bits/stdc++.h>
using namespace std;

long long cantheive(long long n){
    long long count = 0;
    for(long long k = 2; ; k++){
        long long k_cube = k*k*k;
        if(k_cube>n){
            break;
        }
        long long a = n/k_cube;
        count += a;
    }
    return count;
}
int main() {
    long long m;
    cin>>m;
    long long low = 1, high = 1e18;
    long long ans = -1;
    while(low<=high){
        long long mid = (low+high)/2;
        if(cantheive(mid)==m){
            ans = mid;
            high = mid-1;
        }
        else if(cantheive(mid)>m){
            high = mid-1;
        }
        else{
            low = mid+1;
      }
    }
    cout<< ans;

    return 0;
}
```

C. Increasing by Modulo

time limit per test: 2.5 seconds
memory limit per test: 256 megabytes

Toad Zitz has an array of integers, each integer is between 0 and $m-1$ inclusive. The integers are $a_1, a_2, \ldots, a_n$.

In one operation Zitz can choose an integer k and k indices $i_1, i_2, \ldots, i_k$ such that $1 \leq i_1 < i_2 < \ldots < i_k \leq n$. He should then change a_{i_j} to $((a_{i_j}+1) \bmod m)$ for each chosen integer i_j. The integer m is fixed for all operations and indices.

Here $x \bmod y$ denotes the remainder of the division of x by y.

Zitz wants to make his array non-decreasing with the minimum number of such operations. Find this minimum number of operations.

Input

The first line contains two integers n and m ($1 \leq n, m \leq 300\,000$) — the number of integers in the array and the parameter m.

The next line contains n space-separated integers $a_1, a_2, \ldots, a_n$ ($0 \leq a_i < m$) — the given array.

Output

Output one integer: the minimum number of described operations Zitz needs to make his array non-decreasing. If no operations required, print 0.

It is easy to see that with enough operations Zitz can always make his array non-decreasing.

Examples

input Copy

```
5 3
0 0 0 1 2
```

output Copy

```
#include <bits/stdc++.h>
using namespace std;
bool canmake(int x,vector<int>&a,int m){
    int needed = 0; //needed = the minimum value that a[i] must reach (after increment) to keep the sequence non-decreasing so far.
    int n = a.size();
    for(int i = 0; i<n; i++){
        if((a[i]+x)<m){
            if(needed>(a[i]+x)){
                return false;
            }
            else{
                needed = max(needed,a[i]);
            }
        }
        else{
            int wrapped = (a[i]+x)%m;
            if(wrapped<needed && a[i]>needed){ //bad—gap between wrapped and non-wrapped parts
                return false;
            }
        }
    }
    return true;
}
int main() {
    int n,m;
    cin>>n>>m;
    vector<int>a(n);
    for(int i = 0; i<n; i++){
        cin>>a[i];
    }
    int low = 0; int high = m-1; int ans = -1;
    while(low<=high){
        int mid = (low+high)/2;
        if(canmake(mid,a,m)){
            ans = mid;
            high = mid-1;
        }
```

Array-

Ques-

Description | Accepted | Editorial | Solutions | Submissions

48. Rotate Image

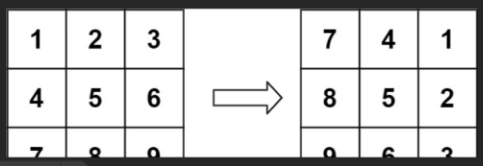
Solved

Medium | Topics | Companies

You are given an `n x n` 2D `matrix` representing an image, rotate the image by **90** degrees (clockwise).

You have to rotate the image **in-place**, which means you have to modify the input 2D matrix directly. **DO NOT** allocate another 2D matrix and do the rotation.

Example 1:



Code

C++ | Auto

```cpp
class Solution {
public:
    void rotate(vector<vector<int>>& matrix) {
        int n = matrix.size();
        int m = matrix[0].size();
        vector<vector<int>>dupmat = matrix;
        for(int i = 0; i<n; i++){
            for(int j = 0; j<m; j++){
                matrix[i][j] = dupmat[n-1-j][i];
            }
        }

    }
};
```

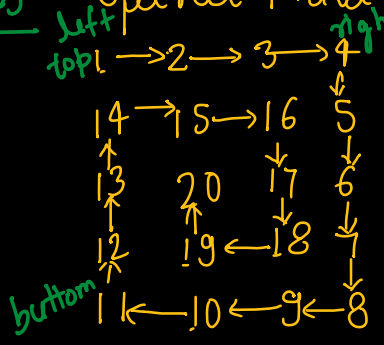
Saved | Ln 9, Col 44

For O(1) space complexity, first take the transpose of the matrix, the reverse each row as follows -

```cpp
class Solution {
public:
    void rotate(vector<vector<int>>& matrix) {
        int n = matrix.size();
        int m = matrix[0].size();
        for(int i = 0; i<n; i++){
            for(int j = i+1; j<m; j++){
                if(i==j)continue;
                else{
                    swap(matrix[i][j],matrix[j][i]);
                }
            }
        }
        for(int i = 0; i<n; i++){
            reverse(matrix[i].begin(), matrix[i].end());
        }

    }
};
```

Ques- Spiral traversal in matrix.



Ans -

```cpp
vector<int> spiralOrder(vector<vector<int>>& matrix) {
    int n = matrix.size();
    int m = matrix[0].size();
    vector<int>ans;
    int left = 0;
    int right = m-1;
    int top = 0;
    int buttom = n-1;
    while(left<=right && top<=buttom){
        for(int i = left; i<=right; i++){
            ans.push_back(matrix[top][i]);
        }
        top++;
        for(int i = top; i<=buttom; i++){
            ans.push_back(matrix[i][right]);
        }
        right--;
        if(top<=buttom){
            for(int i = right; i>=left; i--){
                ans.push_back(matrix[buttom][i]);
            }
            buttom--;
        }
        if(left<=right){
            for(int i = buttom; i>=top; i--){
                ans.push_back(matrix[i][left]);
            }
            left++;
        }
    }
    return ans;
```

Ques- Number of Subarrays with sum K.

arr[] = {1, 2, 3, -3, 1, 1, 1, 4, 2, -3}, K = 3.

We store the prefixsum with their count, using hash map, if at a index sum is S, then we check whether we can get sum S-K from the previous indices. If yes we add the corresponding value of S-K (in hash map) to the count.

(So we need to remove and will add the number of ways of removing S-K to the count).

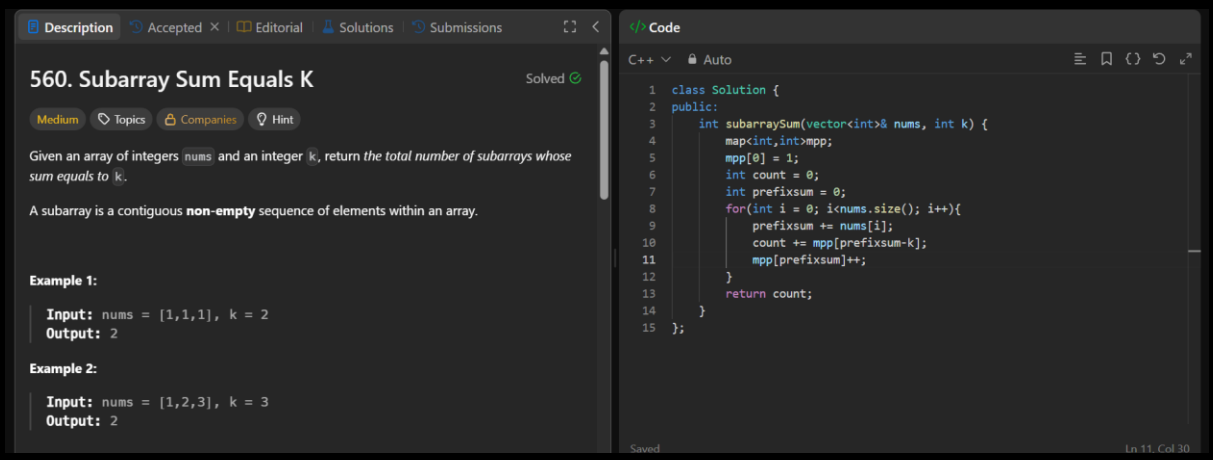
prefixsum=0 1 2 3 3 4 6 8 :

count = 0 1 2 3

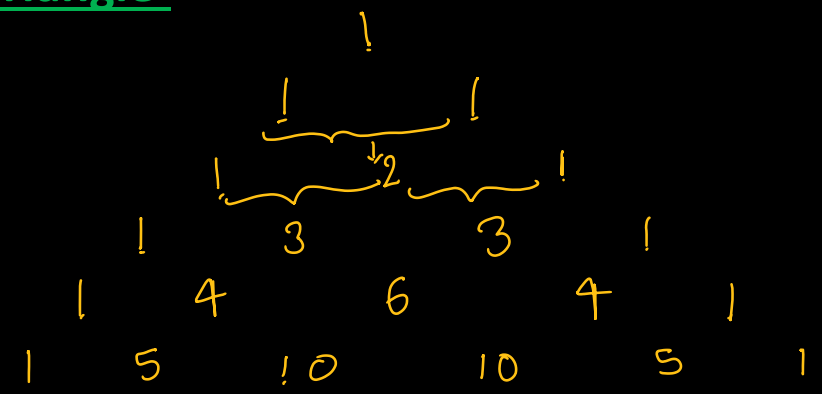
12→1 9→1 .
10→1
5→1
4→1
6→1 2 → now no. of occurances of 6-3 = 3 is 1, so add 1 to the count cus it is the prefixsum number of ways we can remove 3 from 6 and make the sum 3
3→2
1→1
0→1

again sum is 3 so 3-3 is 0 (S-K)
So we add no. of occurances of 0 to the count.

⟶ means for every prefixsum we add corresponding value of S-K to the count.



Pascal Triangle-

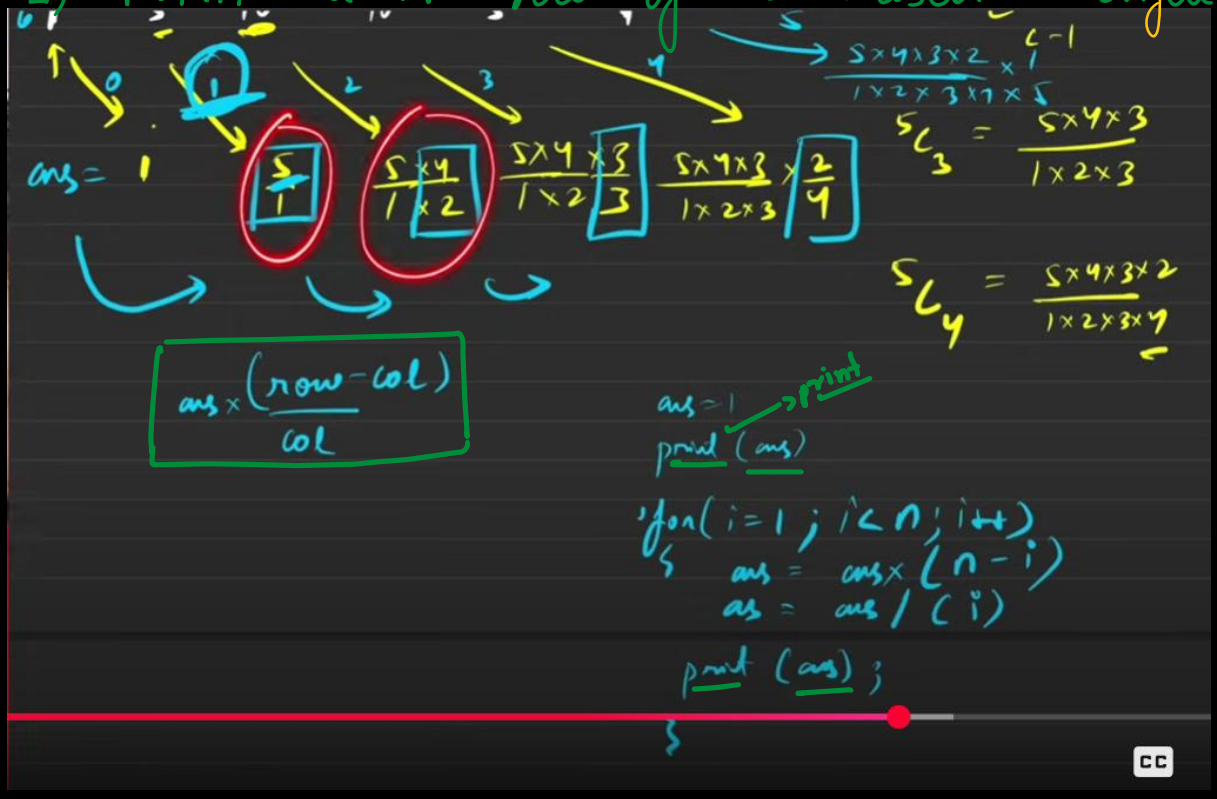


1)-for a Given row and column number, find the element at that place. $^{r-1}C_{c-1}$, where r = given row number, c = given column number

Function to find $^{n}C_{r}$ — $^{5}C_{2} = \frac{5\times4}{2\times1}$

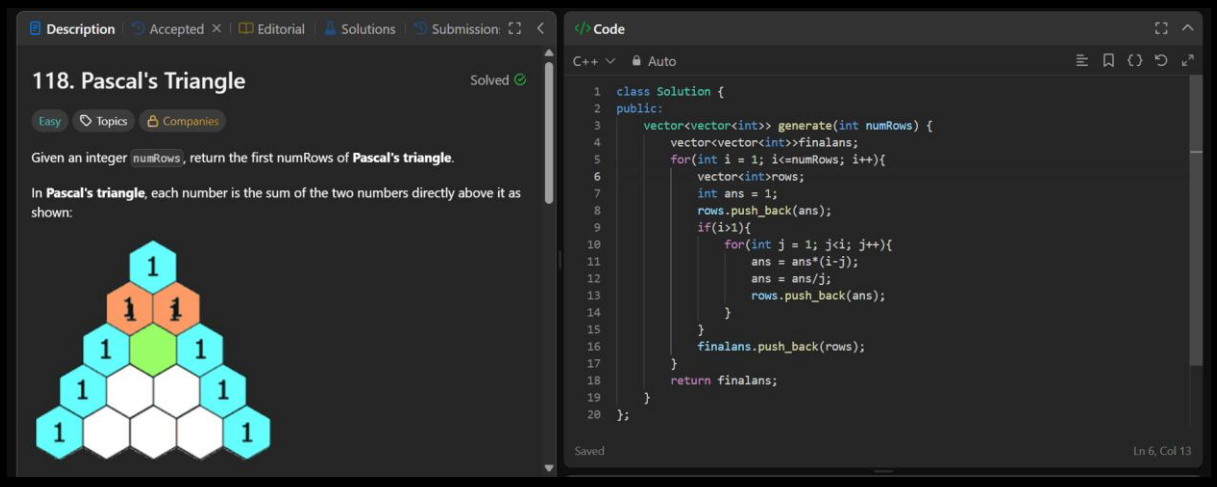
```
ncr(n, r) {
    int res;
    for(int i=0; i<r; i++) {
        res = res*(n-i);
        res = res/i+1;
    }
}
```

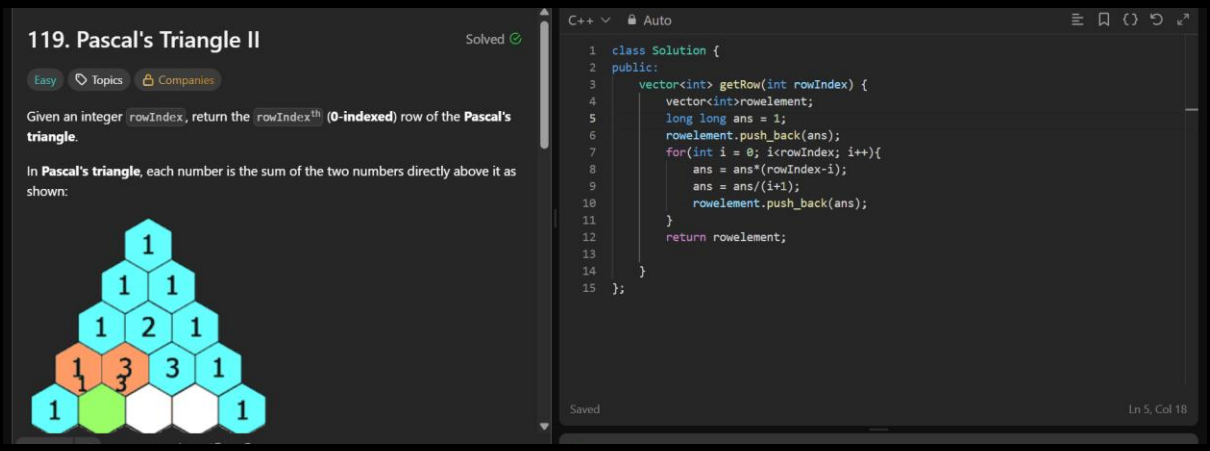
2)- Print the N^{th} row of the Pascal Triangle -



Ques . For a given N → No: of rows, print the entire pascal triangle.

Ans . We will generate each row using above method.





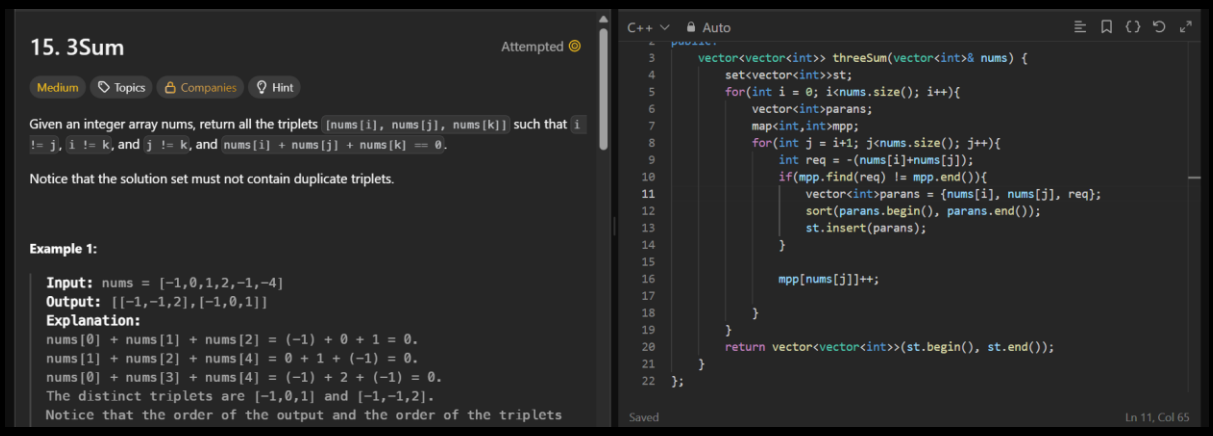
Majority Element (appearing >= n/3 times)- Similar to majority element > n/2.

3 sum problem: We have an array, we need to find the three elements of that array such that the sum of those elements should be zero and they belong to different indices. We need to return those triplets in sorted order.

Method -1- We can solve in $O(n^3)$ complexity using for loop.

Method-2. [-1, 0, 1, 2, -1, 4]

Using two for loops and using a hash map. we will check whether the value $x = -(arr[i] + arr[j])$ is in the map or not. if it is in the map we will insert all the three values in our answer. We will clear the map after each i change. (Basically we are figuring out whether a exist between i & j or not.



15. 3Sum

Attempted

Medium | Topics | Companies | Hint

Given an integer array nums, return all the triplets `[nums[i], nums[j], nums[k]]` such that `i != j`, `i != k`, and `j != k`, and `nums[i] + nums[j] + nums[k] == 0`.

Notice that the solution set must not contain duplicate triplets.

Example 1:

```
Input: nums = [-1,0,1,2,-1,-4]
Output: [[-1,-1,2],[-1,0,1]]
Explanation:
nums[0] + nums[1] + nums[2] = (-1) + 0 + 1 = 0.
nums[1] + nums[2] + nums[4] = 0 + 1 + (-1) = 0.
nums[0] + nums[3] + nums[4] = (-1) + 2 + (-1) = 0.
The distinct triplets are [-1,0,1] and [-1,-1,2].
Notice that the order of the output and the order of the triplets
```

C++ | Auto

```cpp
public:
    vector<vector<int>> threeSum(vector<int>& nums) {
        set<vector<int>>st;
        for(int i = 0; i<nums.size(); i++){
            vector<int>parans;
            map<int,int>mpp;
            for(int j = i+1; j<nums.size(); j++){
                int req = -(nums[i]+nums[j]);
                if(mpp.find(req) != mpp.end()){
                    vector<int>parans = {nums[i], nums[j], req};
                    sort(parans.begin(), parans.end());
                    st.insert(parans);
                }

                mpp[nums[j]]++;

            }
        }
        return vector<vector<int>>(st.begin(), st.end());
    }
};
```

Saved | Ln 11, Col 65

Method-3 – We will use three pointers, i, j, k after sorting the array.

arr = [-2, -2, -2, -1, -1, -1, 0, 0, 0, 2, 2, 2]
i j k

If arr[i] + arr[j] + arr[k] > 0, k--

" " " < 0, j++

" " " = 0, i++ & j = i+1, k = n-1

If on changing value of i, j, k, arr[i] or arr[j] or arr[k] remain same further change their value in the same direction (in order to avoid duplicate & refrain from using extra spaces).

```cpp
vector<vector<int>> threeSum(vector<int>& nums) {
    vector<vector<int>>ans;
    sort(nums.begin(), nums.end());
    int n = nums.size();
    for(int i = 0; i<n ; i++){
        if(i>0 && nums[i]==nums[i-1])continue;
        int j = i+1;
        int k = n-1;
        while(j<k){
            int sum = nums[i]+nums[j]+nums[k];
            if(sum==0){
                vector<int>temp = {nums[i], nums[j],nums[k]};
                ans.push_back(temp);
                j++;
                k--;
                while(j<k && nums[j]==nums[j-1]){
                    j++;
                }
                while(j<k && nums[k]==nums[k+1]){
                    k--;
                }
            }
            else if(sum>0){
                k--;
            }
            else{
                j++;
            }
        }
    }
    return ans;
```

4-SUM PROBLEM-

Batter solution- [1, 2, -2, -1, 0, 1, 2]
i j. K

Now put everything between j & k in the map (similar to 3 sum)

Optimal Solution- We were fixing i & moving J & K in 3-sum. Here we will fix i & j & will move. K & l.

```cpp
    vector<vector<int>> fourSum(vector<int>& nums, int target) {
        sort(nums.begin(), nums.end());
        int n = nums.size();
        vector<vector<int>>ans;
        for(int i = 0; i<n; i++){
            if(i>0 && nums[i]==nums[i-1])continue;
            for(int j = i+1; j<n; j++){
                if(j>i+1 && nums[j]==nums[j-1])continue;
                int k = j+1;
                int l = n-1;
                while(k<l){
                    long long sum = nums[i] + nums[j];
                    sum += nums[k];
                    sum += nums[l];
                    if(sum<target){
                        k++;
                    }
                    else if(sum>target){
                        l--;
                    }
                    else{
                        vector<int>v = {nums[i], nums[j], nums[k], nums[l]};
                        ans.push_back(v);
                        k++;
                        l--;
                        while(k<l && nums[k]==nums[k-1]){
                            k++;
                        }
                        while(k<l && nums[l]==nums[l+1]){
                            l--;
                        }
```

```cpp
                        l--;
                    }
                }
            }
        }
        return ans;
    }
};
```

Count subarrays with XOR as K-

56. Merge Intervals

Medium | Topics | Companies

Given an array of `intervals` where `intervals[i] = [start_i, end_i]`, merge all overlapping intervals, and return *an array of the non-overlapping intervals that cover all the intervals in the input.*

Example 1:

```
Input: intervals = [[1,3],[2,6],[8,10],[15,18]]
Output: [[1,6],[8,10],[15,18]]
Explanation: Since intervals [1,3] and [2,6]
overlap, merge them into [1,6].
```

Merge Overlapping Subintervals-

Intervals (1,3) & (2,6) is overlapping as 2<3, so, can be merged and make (1,6);

Method-1- Can be done in $(O(n^2))$. (after sorting the array).

Method-2.

```cpp
#include<bits/stdc++.h>
vector<vector<int>> mergeOverlappingIntervals(vector<vector<int>> &arr){
    int n = arr.size();
    sort(arr.begin(), arr.end());
    vector<vector<int>> ans;
    for(int i = 0;i<n;i++) {
        if(ans.empty() || arr[i][0] > ans.back()[1]) {
            ans.push_back(arr[i]);
        }
        else {
            ans.back()[1] = max(ans.back()[1], arr[i][1]);
        }
    }
    return ans;
}
```

Merge two sorted arrays without extra space:

arr1 = [1, 3, 5, 7], arr2 = [0, 2, 6, 8, 9]

So, we have to return, arr1 = [0, 1, 2, 3], arr2 = [5, 6, 7, 8, 9]

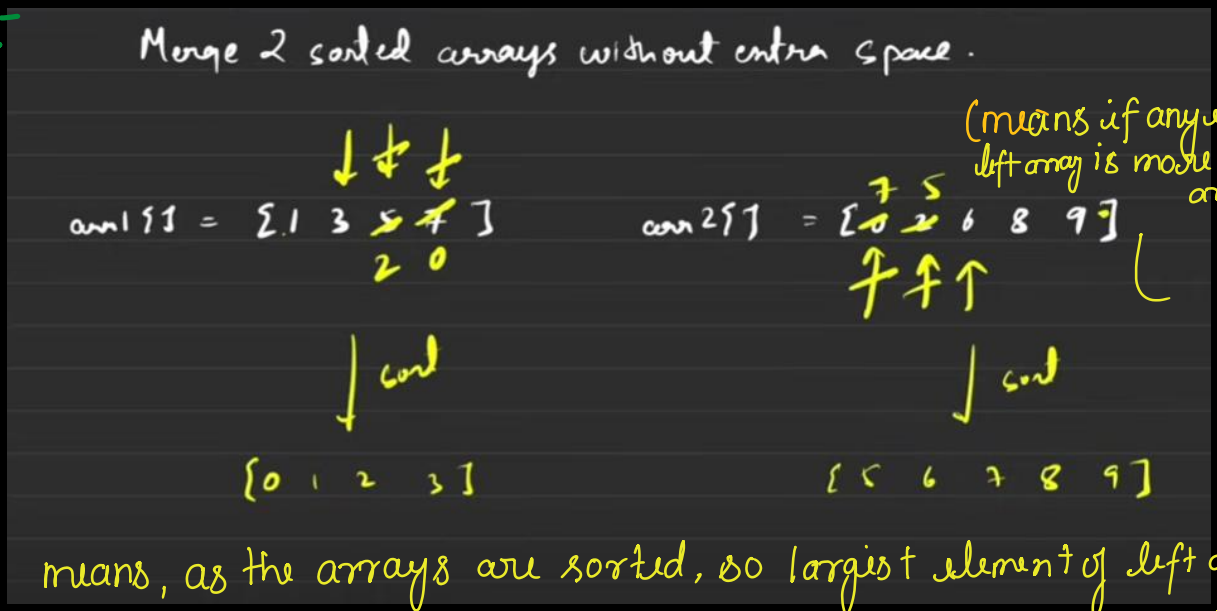
Brute

```cpp
class Solution {
  public:
    void mergeArrays(vector<int>& a, vector<int>& b) {
        // code here
        int i =0;
        int j = 0;
        vector<int>nums;
        int n = a.size();
        int m = b.size();
        while(i<n && j<m){
            if(a[i]<b[j]){
                nums.push_back(a[i]);
                i++;
            }
            else if(a[i]==b[j]){
                nums.push_back(a[i]);
                nums.push_back(b[j]);
                i++;
                j++;
            }
            else{
                nums.push_back(b[j]);
                j++;
            }
        }
        if(i<n){
            for(int k = i; k<n; k++){
                nums.push_back(a[k]);
            }
        }
```

```
        if(j<m){
            for(int k = j; k<m; k++){
                nums.push_back(b[k]);
            }
        }
        for(int x = 0; x<n; x++){
            a[x] = nums[x];
        }
        for(int y = n; y<n+m; y++){
            b[y-n] = nums[y];
        }

    }
};
```

Optimal-



means, as the arrays are sorted, so largest element of left array should be smaller than smallest element of the right array in our answer.

Optimal Solution-2 - Gap method -

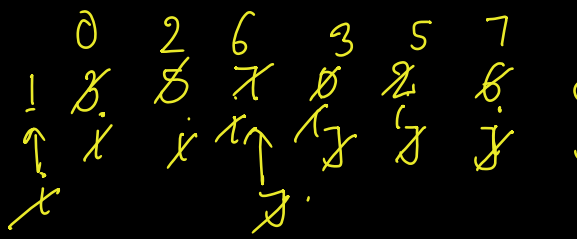
arr1 = [1, 3, 5, 7] arr2 = [0, 2, 6, 8, 9]

$gap = \lceil \frac{4+5}{2} \rceil = \lceil 4.5 \rceil = 5$.

1 3 5 7 0 2 6 8 9

i (and j will be 5 indices ahead), if(arr1[i] < arr2[j]) i++; j++;

once j reaches rightmost, stop. change the gap to gap/2. & again start i:=0 & $j = i + \frac{gap}{2}$



keep this doing untill gap becomes 1.

Set Mismatch-

645. Set Mismatch

Attempted

Easy | Topics | Companies

You have a set of integers `s`, which originally contains all the numbers from `1` to `n`. Unfortunately, due to some error, one of the numbers in `s` got duplicated to another number in the set, which results in **repetition of one** number and **loss of another** number.

You are given an integer array `nums` representing the data status of this set after the error.

Find the number that occurs twice and the number that is missing and return *them in the form of an array.*

Example 1:

```
Input: nums = [1,2,2,4]
Output: [2,3]
```

Butter Solution- Take a hash array from 1 to n, and increase the value at index by 1 if it appears in the array.
Missing number will be the index at which value is zero & repeat number will be the index at which value is 2.

```
3     vector<int> findErrorNums(vector<int>& nums) {
4         int n = nums.size();
5         vector<int>arr(n+1,0);
6         for(int i = 0; i<n; i++){
7             arr[nums[i]]++;
8         }
9         int repeat= 0;
10        int missing = 0;
11        for(int i = 1; i<=n ; i++){
12            if(arr[i]==0)missing = i;
13            if(arr[i]==2)repeat = i;
14        }
15        return vector<int>{repeat,missing};
16    }
17 };
```

Another approach - take the sum of all array elements.
take the sum of all numbers from 1 to n.

arr1 = [6, 4, 3, 2, 1, 1]. , $S_1 = 17$, x = repeat number

$S_2 = Sum_{1 \to 6} = \frac{6 \times 7}{2} = 21$. y = missing

$S_1 - S_2 = [6+4+3+2+1+1] - [1+2+3+4+5+6]$

$= -4$

$\Rightarrow x - y = -4$ —— ①

$[6^2+4^2+3^2+2^2+1^2+1^2] - [1^2+2^2+3^2+4^2+5^2+6^2]$

$\Rightarrow x^2 - y^2 = -24$

$x+y = \frac{+24}{-4} = 6$ —— ②

Add ① & ②

$2x = 2$

$\boxed{x=1}$, $y = 5$

```
3       vector<int> findErrorNums(vector<int>& nums) {
4           int n = nums.size();
5           int repeatnumber = -1;
6           int missingnumber = -1;
7           long long s1 = 0;
8           long long s3 = 0;
9           long long s4 = n*(n+1);
10          s4 = s4*(2*n+1);
11          s4 = s4/6;
12          for(int i = 0; i<n; i++){
13              s1 += nums[i];
14              s3 += nums[i]*nums[i];
15          }
16          long long s2 = n*(n+1)/2;
17          int diff1 = s1-s2;
18          int diff2 = s3-s4;
19          repeatnumber = (diff1 + (diff2/diff1))/2;
20          missingnumber = (diff2/diff1)-repeatnumber;
21          return {repeatnumber,missingnumber};
22      }
23  };
```

Ques -

Description | Accepted | Editorial | Solutions | Submissions

80. Remove Duplicates from Sorted Array II

Solved

Medium | Topics | Companies

Given an integer array `nums` sorted in **non-decreasing order**, remove some duplicates **in-place** such that each unique element appears **at most twice**. The **relative order** of the elements should be kept the **same**.

Since it is impossible to change the length of the array in some languages, you must instead have the result be placed in the **first part** of the array `nums`. More formally, if there are `k` elements after removing the duplicates, then the first `k` elements of `nums` should hold the final result. It does not matter what you leave beyond the first `k` elements.

Return `k` *after placing the final result in the first* `k` *slots of* `nums`.

Do **not** allocate extra space for another array. You must do this by **modifying the input array in-place** with O(1) extra memory.

Custom Judge:

The judge will test your solution with the following code:

Code

C++ | Auto

```cpp
int removeDuplicates(vector<int>& nums) {
    int i = 0;
    int ele = nums[0];
    int count = 1;
    for(int j = 1; j<nums.size(); j++){
        if(nums[j]==ele){
            count++;
            if(count<3){
                i++;
                nums[i] = ele;
            }
        }
        else{
            ele = nums[j];
            count = 1;
            i++;
            nums[i] = ele;
        }
    }
    return i+1;
}
```

Saved | Ln 19, Col 31

Ques -

2419. Longest Subarray With Maximum Bitwise AND

Medium | Topics | Companies | Hint

You are given an integer array `nums` of size `n`.

Consider a **non-empty** subarray from `nums` that has the **maximum** possible **bitwise AND**.

- In other words, let `k` be the maximum value of the bitwise AND of **any** subarray of `nums`. Then, only subarrays with a bitwise AND equal to `k` should be considered.

Return *the length of the* ***longest*** *such subarray.*

The bitwise AND of an array is the bitwise AND of all the numbers in it.

A **subarray** is a contiguous sequence of elements within an array.

C++ | Auto

```cpp
class Solution {
public:
    int longestSubarray(vector<int>& nums) {
        int maxNum = *max_element(nums.begin(), nums.end());
        int count = 0, maxCount = 0;
        for(int i = 0; i < nums.size(); i++){
            if(nums[i] == maxNum){
                count++;
                maxCount = max(maxCount, count);
            } else {
                count = 0;
            }
        }
        return maxCount;
    }
};
```